Tuesday, May 26, 2026

Neural Networks Notes

- neural networks are a group of models that involve several computational layers
- employ nonlinearity between each layer —> allows broader scale of expressivity
- multiple layers compute intermediate representations / high level features of data before arriving at a final answer
- different types of NN's exist
    - feed forward NN's
        - acyclic, nodes partitioned into layers, nodes in layer i+1 receive output of nodes in layer i; outputs never passed back to lower layers
    - convolutional NNs
    - recurrent NN's (have cycles)
    - transformers (use attention)
- gradient of loss uses an algorithm called *backpropogation*
- inspired by brain processes
- Anatomy of a neural network
    - artificial neuron — main component
    - receives any number of scalar inputs (think— dendrites) and outputs one scalar output (axon) which can be used as input for different neuron
    - has tunable weights vector, and might also have nonlinear activation function: if the input vector is **x** and the weight vector is **w** and the function is *f*, then we output $y = f(\vec{\mathbf{w}} \cdot \vec{\mathbf{x}})$ (or can add bias term)
    - some common activation functions are the sigmoid function, the hyperbolic tangent function, and the rectified linear unit (ReLU). ReLU is cheap to compute and avoids vanishing gradient problem that sigmoid has
        - vanishing gradient: gradient becomes extremely small—> learning to slow
    - nodes are connected with directed edges

- each edge has a trainable parameter called its weight
- each node has activation function + parameters
- input nodes given input values —> each node produces its output using all the values produced by all the nodes that have edges directed to it
- there are nodes in the input layer, notes in the hidden layers, and nodes in the output layers

- Why NNs?
  - classical machine learning is linear — what if want to separate points organized as one inner circle and one outer circle? Regression and PCA can't do this. Deep learning is nonlinear
  - example: a single neuron is linear. A single neuron—and indeed any linear system —cannot represent the XOR system (FF—>F, TT—> F, TF—> T, FT—>T). No weights will accomplish this. But with multiple layers of neurons we can accomplish this
  - nonlinear activation function transforms the input space into a space where the XOR function is linearly separable
  - can use neural networks for multi classic classification (number of neurons in the output layer = number of classes)
- Backpropagation
  - to minimize the loss function and attune the edge weights, want to use gradient decent on the loss function. But when neural networks get large, this is too computationally expensive. Backpropagation is a way to calculate the gradients (with respect to the parameters) effiecntly — number of computational steps is linear in the size of the neural network
  - perform computation from the output layer to the input layer
  - use the chain rule to employ the gradient computed in a higher layer to compute that of a lower layer
  - the parameters we are trying to attune are the weights of the edge
  - start with computing Jacobian out final layer, and use this Jacobian to obtain the Jacobian of the last hidden layer, and so on

    - loss function for deep neural nets is non-convex with respect to the parameters of the network —> gradient descent not guaranteed to find the minimizer—> choice of initial parameters matters a lot
    - think of it as assigning blame
    - if loss is high, adjust weights a lot; if loss is low, only tweak a little bit
- in practice, neural network designer chooses number of layers, number of hidden units, and activation functions
- like all ML models, we have to deal with overfitting, bias-variance tradeoff, regularization
- exploding gradients can be a problem—when gradients become too large and can become unstable. neural networks repeatedly multiply gradients, to exploding and vanishing gradients can be more of a problem —> more instability. important to attune learning rate appropriately; be careful in initializing weight. ReLU activation function doesn't saturate as much as sigmoid and tanh — meaning that we don't have have vanishing gradients. Can also use "batch normalization"=normalizing inputs to a layer to have zero mean and unit variance; or "gradient clipping": which applies a maximum threshold to gradients to prevent instability
- application of NNs
    - computer vision, including image recognition and autonomous vehicles (convolution neural networks)
    - natural language processing (transformers): machine translation and chatbots
    - speech recognition (RNNs and deep nets): transcription and voice assistants
    - forecasting and time series
    - pattern recognition

My sources: Princeton University COS324 Course Notes, Google For Developers Machine Learning, IBM "What is a Neural Network?", Geeks for Geeks "Vanishing and Exploding Gradients Problems in Deep Learning"